C Implementation

This appendix documents the implementation of two service layers used by SpisNemt: the Firestore-backed UserService which handles user profile creation, preferences, and saved recipes and the HTTP client MealDBService which handles recipe lookup and lists from TheMealDB.

C.1 C4 Model

As described in section 7.2, an analog C4 model was created and iterated on a whiteboard. This model was implemented using structurizr, as it is the recommended approach stated in the official C4 documentation. Structurizr uses a Diagrams as code approach where instead of dragging and dropping boxes and arrows on a canvas, they are programmed.

The process of implementing the whiteboard C4 model in structurizr was straightforward after figuring out the Structurizr DSL language. Structurizr DSL programmatically defines architecture, and it follows a hierarchal structure where a *model* acts as the single source of truth. Elements are defined within a nested scope, and the *Software System* object serves as the parent encapsulating *Container* objects, which in turn encapsulate *Component* objects. The structure can be seen in figure 1.

workspace.dsl

```
workspace "My System" {

  model {
    user = person "User"{
      tags "FontStyle"
    }

    SpisNemtApp  = softwareSystem "SpisNemtApp" {
      tags "FontStyle"

      UI = container "UI" {
        tags "FontStyle", "Boundary"
          HomeScreen = component "Home\nScreen" {
            tags "FontStyle"
          }

          ExploreScreen = component "Explore\nScreen" {
            tags "FontStyle"
          }

          SavedRecipesScreen = component "SavedRecipes\nScreen" {
            tags "FontStyle"
          }

          AccountScreen = component "Account\nScreen" {
            tags "FontStyle"
          }

          LoginScreen = component "Login\nScreen" {
            tags "FontStyle"
          }

          CreateAccountScreen = component "CreateAccount\nScreen" {
            tags "FontStyle"
          }
      }

      // Other containers and components

    }

    theme default

  }
}
```

Figure 1: C4 code snippet written in Structurizr DSL within *workspace.dsl*, illustrating the hierarchical nesting of actors, system parameters, containers, and isolated UI components.

C.2 UserService

UserService is a folder in SpisNemt which groups together six files, where each file is responsible for a single function call which contacts Firestore. Additionally, two files named *types.ts* and *refs.ts* lies within the *UserService* folder. The file *types.ts* declares two interfaces, *UserIdentity* and *UserProfileDocument* as seen in figure 2. UserIdentity

is used for the user account information and UserProfileDocument is used for properties such as the users preferences or saved recipes.

types.ts

```ts
export interface UserIdentity {
  id: string;
  email: string;
  name: string | null;
}

export interface UserProfileDocument {
  email: string;
  name: string | null;
  preferences: {
    area: string[];
    category: string[];
  };
  savedRecipes: number[];
  updatedAt?: unknown;
}

```

Figure 2: types.ts declaring the two interfaces used by UserService. UserIdentity is used for the user account information and UserProfileDocument is used for properties such as the users preferences or saved recipes.

refs.ts exports the helper *userProfileDocRef(userId)* which constructs and returns a Firestore *DocumentReference* for the path *users/userId*. It only builds the reference object; callers such as *getDoc* and *setDoc* use that reference to read, write or listen to the logged-in user's profile.

refs.ts

```
4 const USERS_COLLECTION = "users";
5
6 export function userProfileDocRef(userId: string) {
7   return doc(db, USERS_COLLECTION, userId);
8 }
9
```

Figure 3: refs.ts exporting *userProfileDocRef(userId)* for construction and returning the Firestore *DocumentReference*.

Previously mentioned, *UserService* contains six files for communication with Firestore. Firstly, *createUserProfile.ts* is used for creating the user profile for an account using the helper function, *userProfileDocRef* from *refs.ts*. When updating preferences *saveUserPreferences* from *saveUserPreferences.ts* is called which uses the *UserIdentity* interface from *types.ts* as well as the DocumentReferences from *refs.ts*. A complete overview of what each file's responsibility can be found i table 1.

Module / Method	Description
`refs.ts`	Exports userProfileDocRef(userId) which returns a Firestore `DocumentReference` for users/{userId}; used by all read/write operations.
`createUserProfile`	Initializes the user document (email, name, empty preferences and savedRecipes).
`getUserProfile`	Reads and returns a user profile if it exists.
`saveUserSavedRecipes`	Merges the provided savedRecipes array into the user document.
`SavedRecipes`	Helper module exposing load/save helpers for saved recipes.
`saveUserPreferences`	Merges preferences (area, category) into the user document.
`Preferences`	Helper module exposing load/save helpers for user preferences.

Table 1: Overview of individual file responsibilities and method behaviors within the *UserService* module.

As shown in table 1, the architecture partitions states into dedicated modules. By isolating reads via *getUserProfile* from updates like *saveUserPreferences*, the service guarantees that components only trigger network requests for the data scope they manage, preventing bloated payloads to Firestore.

C.3 MealDBService

MealDBService is a folder which contains communication centered around TheMealDB HTTP API. Each file contains a function which issues a *fetch()* to a specific endpoint and returns recipes. An overview of what each file's function does is seen in table 2. Some fetches uses a *v1* endpoint while others use a paid *v2* endpoint.

Module / Method	Description
`getRandomRecipe`	Fetches one random recipe
`get10RandomRecipes`	Fetches an array of ten random recipes
`getRecipeDetailsById`	Fetches recipes based on ID
`getRecipesByMultiIngredients`	Fetches recipes based on comma separated ingredients. Multiple ingredients can be used.
`getAllCategories`	Fetches an array of all categories of food
`getAllCuisines`	Fetches an array of all cuisines.

Table 2: Overview of fetch methods, endpoint targets, and behaviors within the *MealDB-Service* module.

The most complex fetch call is *getRecipesByMultiIngredients* where an array of ingredients is used as a parameter. This parameter gets comma separated before being used as parameter in the actual fetch call. The implementation of *getRecipesByMultiIngredients* is seen in figure 4. This fetch uses the users available ingredients, which is scanned with a camera or input with text in *exploreScreen*, as parameter.

getRecipesByMultiIngredients.ts

```
5  export const getRecipesByMultiIngredients = async (ingredients: string[]) => {
6    try {
7      const query = ingredients.join(",");
8      const response = await fetch(`${BASE_URL}/${API_KEY}/filter.php?i=${query}`);
9
10     if (!response.ok) {
11       throw new Error(`HTTP error! status: ${response.status}`);
12     }
13
14     const data = await response.json();
15     return data.meals ?? [];
16   } catch (error) {
17     console.error("Error fetching recipes by ingredients:", error);
18     throw error;
19   }
20 };
```

Figure 4: Implementation of *getRecipesByMultiIngredients.ts* showing how the *ingredients* array is formatted into a comma-separated query string and dispatched to TheMealDB API with robust asynchronous error handling.